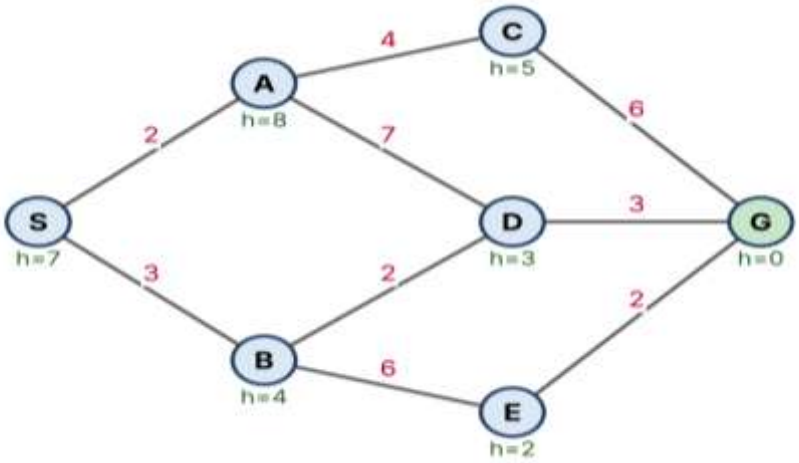
QUESTION 1: GREEDY BEST-FIRST SEARCH

Greedy best-first search always expands the open node with the smallest heuristic value h(n); it ignores the cost already paid g(n). Start S (h = 7), goal G. Closed nodes are not re-added.



(a) Expansion table

Step	Node expanded	Open list after expansion (node: h)	Remark
0	-	S:7	Initial open list
1	S	B:4, A:8	Add neighbours A(8), B(4)
2	B	E:2, D:3, A:8	Add D(3), E(2); S is closed
3	E	G:0, D:3, A:8	Add G(0)
4	G	(goal)	G has the lowest h, so it is selected: goal reached

(b) Path found and cost

Path: S -> B -> G -> E

Cost: 3+6+2 = 11

(c) Optimal?

No. Another path, S -> B -> D -> G, costs 3 + 2 + 3 = 8, which is less than 11. Greedy picked E at step 3 because h(E) = 2 < h(D) = 3, but it ignored that reaching E cost 9 whereas D cost only 5, and the edge E-G (2) is cheaper than D-G (3) by just 1. Because greedy uses only h(n) and not g(n), it is fast but neither optimal nor guaranteed to find the cheapest path.

Program:

```
import heapq
graph = {
    'S': {'A': 2, 'B': 3},
    'A': {'S': 2, 'C': 4, 'D': 7},
    'B': {'S': 3, 'D': 2, 'E': 6},
    'C': {'A': 4, 'G': 6},
    'D': {'A': 7, 'B': 2, 'G': 3},
    'E': {'B': 6, 'G': 2},
    'G': {'C': 6, 'D': 3, 'E': 2},
}
h = {'S': 7, 'A': 8, 'B': 4, 'C': 5, 'D': 3, 'E': 2, 'G': 0}
def greedy(start, goal):
    open_list = [(h[start], start, [start], 0)]
    closed = set()
    step = 0
    while open_list:
        hn, node, path, g = heapq.heappop(open_list)
        if node in closed:
            continue
        step += 1
        closed.add(node)
        if node == goal:
            print(f"Step {step}: expand {node} -> GOAL reached")
            return path, g
        for nb, c in graph[node].items():
            if nb not in closed:
                heapq.heappush(open_list, (h[nb], nb, path + [nb], g + c))
        show = sorted({(n, hh) for hh, n, _, _ in open_list if n not in closed}, key=lambda x: (x[1], x[0]))
        print(f"Step {step}: expand {node:2s} open = {[f'{n}:{hh}' for n, hh in show]}")
    return None, None
print("=== Greedy Best-First Search ===")
p, c = greedy('S', 'G')
print("Path:", ' -> '.join(p), "| cost =", c)
```

Program output:

```
=== Greedy Best-First Search ===
Step 1: expand S  open = ['B:4', 'A:8']
Step 2: expand B  open = ['E:2', 'D:3', 'A:8']
Step 3: expand E  open = ['G:0', 'D:3', 'A:8']
Step 4: expand G -> GOAL reached
Path: S -> B -> E -> G | cost = 11
>>>
```

QUESTION 2: A* SEARCH

A* expands the node with the smallest f(n) = g(n) + h(n). Open list entries are written as (node: g + h = f).

(a) Expansion table

Step	Node expanded	Open list after expansion (node: g + h = f)
0	-	S: 0+7 = 7
1	S (f=7)	B: 3+4 = 7, A: 2+8 = 10
2	B (f=7)	D: 5+3 = 8, A: 2+8 = 10, E: 9+2 = 11
3	D (f=8)	G: 8+0 = 8, A: 2+8 = 10, E: 9+2 = 11 (A via D: g=12, worse than 2, discarded)
4	G (f=8)	(goal) - lowest f in the open list, so A* stops

(b) Path, cost and comparison with Greedy Best First Search

Path: S - > B -> D - > G
Cost: 3+2+3 = 8

	Greedy (Q1)	A* (Q2)
Path	S-B-E-G	S-B-D-G
Cost	11	8
Nodes expanded	4 (S, B, E, G)	4 (S, B, D, G)
Optimal?	No	Yes

Both expanded four nodes, but A* found the cheaper path (8 versus 11) because it combines the cost so far with the estimate. At step 2, E scored 9+2 = 11, which is worse than D at 5+3 = 8, so A* correctly avoided the E route.

(c) Admissibility check

h(n) is admissible if h(n) <= h*(n), where h*(n) is the true cheapest remaining cost to G.

Node	h(n)	True remaining cost h*(n)	Cheapest route	h <= h*?
S	7	8	S-B-D-G = 3+2+3	Yes
A	8	10	A-C-G = 4+6 (or A-D-G = 7+3)	Yes
B	4	5	B-D-G = 2+3	Yes
D	3	3	D-G	Yes (equal)
C	5	6	C-G	Yes (extra)
E	2	2	E-G	Yes (extra)

Every node satisfies h <= h*, so h is admissible (it never overestimates). It is also consistent: for every edge, h(n) <= cost(n, n') + h(n'), for example h(S) = 7 <= 3 + h(B) = 7.

Program:

```
import heapq
graph = {
    'S': {'A': 2, 'B': 3},'A': {'S': 2, 'C': 4, 'D': 7},
    'B': {'S': 3, 'D': 2, 'E': 6},
    'C': {'A': 4, 'G': 6},
    'D': {'A': 7, 'B': 2, 'G': 3},
    'E': {'B': 6, 'G': 2},
    'G': {'C': 6, 'D': 3, 'E': 2},
}
h = {'S': 7, 'A': 8, 'B': 4, 'C': 5, 'D': 3, 'E': 2, 'G': 0}
def astar(start, goal):
    open_list = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    closed = set()
    step = 0
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node in closed:
            continue
        step += 1
        closed.add(node)
        if node == goal:
            print(f"Step {step}: expand {node} -> GOAL reached (f={f})")
            return path, g
        for nb, c in graph[node].items():
            ng = g + c
            if nb not in closed and ng < best_g.get(nb, float('inf')):
                best_g[nb] = ng
                heapq.heappush(open_list, (ng + h[nb], ng, nb, path + [nb]))
        live = {}
        for f2, g2, n, _ in open_list:
            if n not in closed and best_g[n] == g2:
                live[n] = (g2, h[n], f2)
        show = sorted(live.items(), key=lambda x: (x[1][2], x[0]))
        print(f"Step {step}: expand {node:2s} open = [{f'{n}:{a}+{b}={c}' for n, (a, b, c) in show}]")
    return None, None
def true_cost(start, goal='G'):
    dist = {n: float('inf') for n in graph}
    dist[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, c in graph[u].items():
            if d + c < dist[v]:
                dist[v] = d + c
                heapq.heappush(pq, (dist[v], v))
    return dist[goal]
print("=== A* Search ===")
p, c = astar('S', 'G')
print("Path:", ' -> '.join(p), "| cost =", c)

print("\n=== Admissibility check ===")
print(f"{'Node':5s}{'h(n)':6s}{'h*(n)':7s}h<=h*?")
for n in 'SABCDEG':
    t = true_cost(n)
    print(f"{n:5s}{h[n]:<6d}{t:<7d}{h[n] <= t}")
```

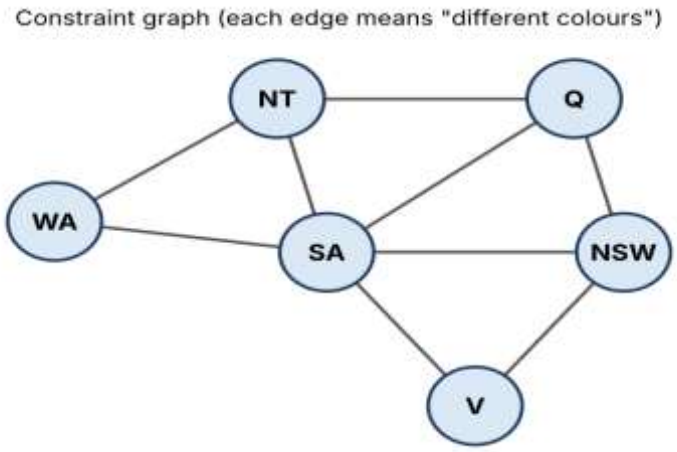
Program Output:

```
=== A* Search ===
Step 1: expand S  open = ['B:3+4=7', 'A:2+8=10']
Step 2: expand B  open = ['D:5+3=8', 'A:2+8=10', 'E:9+2=11']
Step 3: expand D  open = ['G:8+0=8', 'A:2+8=10', 'E:9+2=11']
Step 4: expand G -> GOAL reached (f=8)
Path: S -> B -> D -> G | cost = 8
```

QUESTION 3: CONSTRAINT SATISFACTION PROBLEM

(a) CSP formulation

- **Variables:** WA, NT, SA, Q, NSW, V (the six regions).
- **Domains:** D(each variable) = {R, G, B}.
- **Constraints:** neighbouring regions must differ: WA != NT, WA != SA, NT != SA, NT != Q, SA != Q, SA != NSW, SA != V, Q != NSW, NSW != V (9 binary constraints).



(b) Backtracking with forward checking, MRV, degree heuristic

Variable order rule: smallest remaining domain (MRV); ties by the most unassigned neighbours (degree); then by the order WA, NT, SA, Q, NSW, V. Values are tried as R, G, B. All domains start as {R, G, B}.

Step	Variable chosen and why	Value	Domains of unassigned regions after forward checking
1	SA: all MRV = 3; SA has the highest degree (5)	R	WA{G,B} NT{G,B} Q{G,B} NSW{G,B} V{G,B}
2	NT: MRV = 2 for all; degree NT = Q = NSW = 2 (WA, V = 1); NT first in the order	G	WA{B} Q{B} NSW{G,B} V{G,B}
3	Q: WA and Q have MRV = 1; degree Q = 1 (NSW), WA = 0	B	WA{B} NSW{G} V{G,B}
4	NSW: MRV = 1; degree NSW = 1 (V), WA = 0	G	WA{B} V{B}
5	WA: WA and V both have MRV = 1 and degree 0; WA first in the order	B	V{B}
6	V: only variable left, domain {B}	B	- (all assigned)

Final Colouring:

WA = Blue, NT = Green, SA = Red, Q = Blue, NSW = Green, V = Blue

(c) MRV and degree help

- **MRV (fail-first):** picking the variable with the fewest legal values exposes dead ends early. Here, after SA = R and NT = G, WA and Q were forced to Blue immediately, so the search followed forced moves instead of guessing, and any contradiction would have shown up within a step or two, pruning large subtrees.
- **Degree heuristic:** when MRV ties (at the start every domain had 3 colours), choosing the variable involved in the most constraints removes the most values from neighbours. Choosing SA first (5 neighbours) shrank five domains at once, which is why the rest of the search was almost deterministic.
- Together with forward checking they kept the search to 6 assignments with zero backtracks, the minimum possible.

Program:

```
variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V']
colors = ['R', 'G', 'B']
edges = [('WA','NT'),('WA','SA'),('NT','SA'),('NT','Q'),('SA','Q'),
         ('SA','NSW'),('SA','V'),('Q','NSW'),('NSW','V')]
nbrs = {v: set() for v in variables}
for a, b in edges:
    nbrs[a].add(b); nbrs[b].add(a)
def select_var(assign, dom):
    unassigned = [v for v in variables if v not in assign]
    return min(unassigned, key=lambda v: (len(dom[v]),
               -sum(1 for n in nbrs[v] if n not in assign), variables.index(v)))
def backtrack(assign, dom, log):
    if len(assign) == len(variables):
        return assign
    var = select_var(assign, dom)
    for val in [c for c in colors if c in dom[var]]:
        new_dom = {v: list(d) for v, d in dom.items()}
        new_dom[var] = [val]
        ok = True
        for n in nbrs[var]:
            if n not in assign and val in new_dom[n]:
                new_dom[n].remove(val)
                if not new_dom[n]:
                    ok = False
        step = len(log) + 1
        rest = {v: ''.join(new_dom[v]) for v in variables if v not in assign and v != var}
        log.append(f"{step}. {var} = {val}   domains: {rest}")
        print(log[-1])
        if ok:
            res = backtrack({**assign, var: val}, new_dom, log)
            if res:
                return res
        print(f"   backtrack from {var}={val}")
    return None
dom = {v: list(colors) for v in variables}
print("Initial domains:", {v: ''.join(d) for v, d in dom.items()})
sol = backtrack({}, dom, [])
print("Final colouring:", sol)
print("Valid:", all(sol[a] != sol[b] for a, b in edges))
```

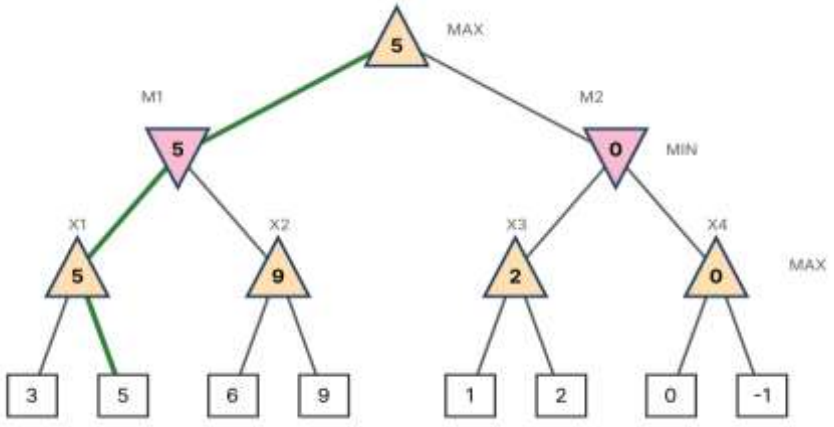
Program Output:

```
Initial domains: {'WA': 'RGB', 'NT': 'RGB', 'SA': 'RGB', 'Q': 'RGB', 'NSW': 'RGB', 'V': 'RGB'}
1. SA = R   domains: {'WA': 'GB', 'NT': 'GB', 'Q': 'GB', 'NSW': 'GB', 'V': 'GB'}
2. NT = G   domains: {'WA': 'B', 'Q': 'B', 'NSW': 'GB', 'V': 'GB'}
3. Q = B   domains: {'WA': 'B', 'NSW': 'G', 'V': 'GB'}
4. NSW = G   domains: {'WA': 'B', 'V': 'B'}
5. WA = B   domains: {'V': 'B'}
6. V = B   domains: {}
Final colouring: {'SA': 'R', 'NT': 'G', 'Q': 'B', 'NSW': 'G', 'WA': 'B', 'V': 'B'}
Valid: True
>>>
```

QUESTION 4: MINIMAX WITH A HEURISTIC EVALUATION FUNCTION

(a) Backed-up values

Leaves (left to right): 3, 5 | 6, 9 | 1, 2 | 0, -1. MAX nodes take the larger child value, MIN nodes the smaller.



Node	Type	Children values	Backed-up value
X1	MAX	3, 5	5
X2	MAX	6, 9	9
X3	MAX	1, 2	2
X4	MAX	0, -1	0
M1	MIN	X1 = 5, X2 = 9	5
M2	MIN	X3 = 2, X4 = 0	0
Root	MAX	M1 = 5, M2 = 0	5

(b) Best first move and principal variation

Principal variation: Root -> M1 -> X1 -> leaf 5. That is, MAX goes left, MIN replies with X1 (since min (5, 9) = 5), and MAX then takes the leaf with value 5. The game value is 5.

(c) Tic-tac-toe evaluation E = (lines open for X) - (lines open for O)

Board (as read from the question): X at top-left, O at top-middle, X at middle-left, O at bottom-left; all other squares empty. A line is "open" for a player if it contains no opponent mark.

	Col 1	Col 2	Col 3
Row 1	X	O	
Row 2	X		
Row 3	O		

Line	Contents	Open for X?	Open for O?
Row 1	X, O, _	No	No
Row 2	X, _, _	Yes	No
Row 3	O, _, _	No	Yes
Column 1	X, X, O	No	No
Column 2	O, _, _	No	Yes
Column 3	_, _, _	Yes	Yes
Diagonal (top-left to bottom-right)	X, _, _	Yes	No
Diagonal (top-right to bottom-left)	_, _, O	No	Yes
Total		3	4

E = 3 – 4 = -1 (Slightly Favourable to O)

Python program

```
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
tree = {
    'root': ('MAX', ['M1', 'M2']),
    'M1': ('MIN', ['X1', 'X2']),
    'M2': ('MIN', ['X3', 'X4']),
    'X1': ('MAX', [3, 5]),
    'X2': ('MAX', [6, 9]),
    'X3': ('MAX', [1, 2]),
    'X4': ('MAX', [0, -1]),
}
values = {}
def minimax(node):
    kind, children = tree[node]
    vals = [c if isinstance(c, int) else minimax(c) for c in children]
    values[node] = max(vals) if kind == 'MAX' else min(vals)
    return values[node]
minimax('root')
for n in ['X1', 'X2', 'X3', 'X4', 'M1', 'M2', 'root']:
    print(f"{n:5s} ({tree[n][0]}) = {values[n]}")
best = max(['M1', 'M2'], key=lambda n: values[n])
print("Best first move for MAX ->", best, "with value", values[best])
board = ['X', 'O', ' ',
         'X', ' ', ' ',
         'O', ' ', ' ']
lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
def evaluate(b):
    open_x = [l for l in lines if all(b[i] != 'O' for i in l)]
    open_o = [l for l in lines if all(b[i] != 'X' for i in l)]
    return open_x, open_o, len(open_x) - len(open_o)
ox, oo, E = evaluate(board)
print("Lines open for X:", len(ox), ox)
print("Lines open for O:", len(oo), oo)
print("E =", len(ox), "-", len(oo), "=", E)
```

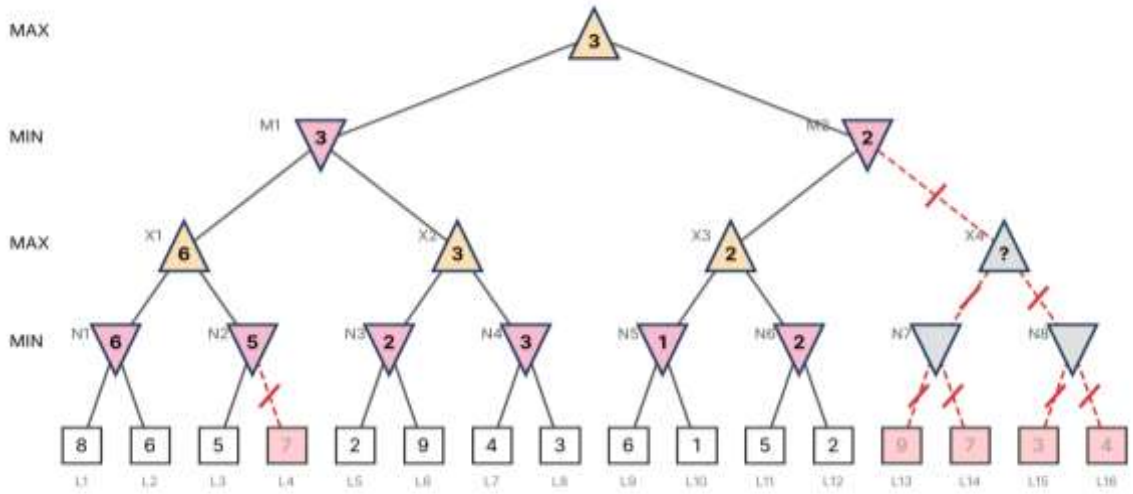
Program output:

```
X1    (MAX) = 5
X2    (MAX) = 9
X3    (MAX) = 2
X4    (MAX) = 0
M1    (MIN) = 5
M2    (MIN) = 0
root  (MAX) = 5
Best first move for MAX -> M1 with value 5
Lines open for X: 3 [(3, 4, 5), (2, 5, 8), (0, 4, 8)]
Lines open for O: 4 [(6, 7, 8), (1, 4, 7), (2, 5, 8), (2, 4, 6)]
E = 3 - 4 = -1
>>>
```

QUESTION 5: ALPHA-BETA PRUNING

(a) Alpha-beta trace

Leaves L1-L16: 8, 6, 5, 7, 2, 9, 4, 3, 6, 1, 5, 2, 9, 7, 3, 4. Levels: Root = MAX, M = MIN, X = MAX, N = MIN. A branch is pruned when alpha >= beta. Initial values are alpha = -inf, beta = +inf.



Node	Type	(alpha, beta) on entry	What happens	Returns
Root	MAX	(-inf, +inf)	Go to M1	3
M1	MIN	(-inf, +inf)	Go to X1	3
X1	MAX	(-inf, +inf)	Go to N1	6
N1	MIN	(-inf, +inf)	L1 = 8 -> beta 8; L2 = 6 -> beta 6	6
X1	MAX	alpha becomes 6	Go to N2 with (6, +inf)	
N2	MIN	(6, +inf)	L3 = 5 -> beta 5 <= alpha 6: PRUNE L4 (7)	5
X1	MAX		value = max(6, 5) = 6	6
M1	MIN	beta becomes 6	Go to X2 with (-inf, 6)	
N3	MIN	(-inf, 6)	L5 = 2 -> beta 2; L6 = 9 (no change); no cut as alpha = -inf	2
X2	MAX	alpha becomes 2	Go to N4 with (2, 6)	
N4	MIN	(2, 6)	L7 = 4 -> beta 4; L8 = 3 -> beta 3 (still > alpha 2, no cut)	3
X2	MAX		value = max(2, 3) = 3	3
M1	MIN		value = min(6, 3) = 3	3
Root	MAX	alpha becomes 3	Go to M2 with (3, +inf)	
M2	MIN	(3, +inf)	Go to X3	
X3	MAX	(3, +inf)	Go to N5	
N5	MIN	(3, +inf)	L9 = 6 -> beta 6; L10 = 1 -> beta 1 (<= 3, but no leaf left)	1

Node	Type	(alpha, beta) on entry	What happens	Returns
N6	MIN	(3, +inf)	L11 = 5 -> beta 5; L12 = 2 -> beta 2	2
X3	MAX		value = max(1, 2) = 2	2
M2	MIN	beta becomes 2	beta 2 <= alpha 3: PRUNE X4 (N7, N8 and leaves L13-L16)	2
Root	MAX		value = max(3, 2) = 3	3

Pruned branches: L4 (value 7) under N2, and the whole subtree X4, i.e. leaves L13-L16 (9, 7, 3, 4).

Value of the root = 3

(b) Leaves evaluated and comparison with minimax

	Plain minimax	Alpha-beta
Leaves evaluated	16	11 (L1-L3, L5-L12)
Leaves skipped	0	5 (L4, L13-L16)
Root value	3	3

Alpha-beta evaluated 11 of the 16 leaves, about 31% fewer than minimax. Both give a root value of 3: pruning only discards branches that provably cannot change the result, so the answer is identical. Plain minimax check: N-values 6, 5, 2, 3, 1, 2, 7, 3; X-values 6, 3, 2, 7; M-values 3, 2; root = max (3, 2) = 3.

(c) Effect of move ordering

Alpha-beta prunes more when the best moves are examined first. If MAX nodes try their best child first and MIN nodes try their lowest-valued child first, the bounds alpha and beta tighten quickly and cutoffs happen as early as possible. In the best case, the nodes examined drop from O(b^d) to about O(b^(d/2)), so the search can go roughly twice as deep in the same time. With the worst ordering (best move last) no pruning occurs and alpha-beta degrades to plain minimax. In this tree, M2 was cut off after only X3 because its value (2) fell below the alpha (3) already established by M1; if M2 had been searched first, that bound would not have existed and fewer branches would have been pruned. In practice, engines use heuristic ordering (captures first, killer moves, iterative deepening) to approach the best case.

Python program

```
leaves = [8, 6, 5, 7, 2, 9, 4, 3, 6, 1, 5, 2, 9, 7, 3, 4]
INF = float('inf')
count = {'ab': 0, 'mm': 0}
def minimax(depth, idx, maximizing):
    if depth == 4:
        count['mm'] += 1
        return leaves[idx]
    vals = [minimax(depth + 1, idx * 2 + i, not maximizing) for i in range(2)]
    return max(vals) if maximizing else min(vals)
def alphabeta(depth, idx, maximizing, alpha, beta, name='root'):
    if depth == 4:
        count['ab'] += 1
        print(f"  leaf #{idx+1:2d} = {leaves[idx]}")
        return leaves[idx]
    kind = 'MAX' if maximizing else 'MIN'
    print(f"{'  '*depth}visit {name} ({kind}) alpha={alpha}, beta={beta}")
    value = -INF if maximizing else INF
    for i in range(2):
        child = alphabeta(depth + 1, idx * 2 + i, not maximizing, alpha, beta, f"{name}.{i+1}")
        if maximizing:
            value = max(value, child); alpha = max(alpha, value)
        else:
            value = min(value, child); beta = min(beta, value)
        if alpha >= beta:
            if i == 0:
                print(f"{'  '*depth}  PRUNE remaining children of {name} (alpha={alpha} >= beta={beta})")
            break
    print(f"{'  '*depth}{name} returns {value}")
    return value
root = alphabeta(0, 0, True, -INF, INF)
mm = minimax(0, 0, True)
print("\nAlpha-beta root value:", root, "| leaves evaluated:", count['ab'])
print("Minimax root value   :", mm, "| leaves evaluated:", count['mm'])
print("Same root value:", root == mm)
```

Program output:

```
Alpha-beta root value: 3 | leaves evaluated: 11
Minimax root value   : 3 | leaves evaluated: 16
Same root value: True
>>>
```